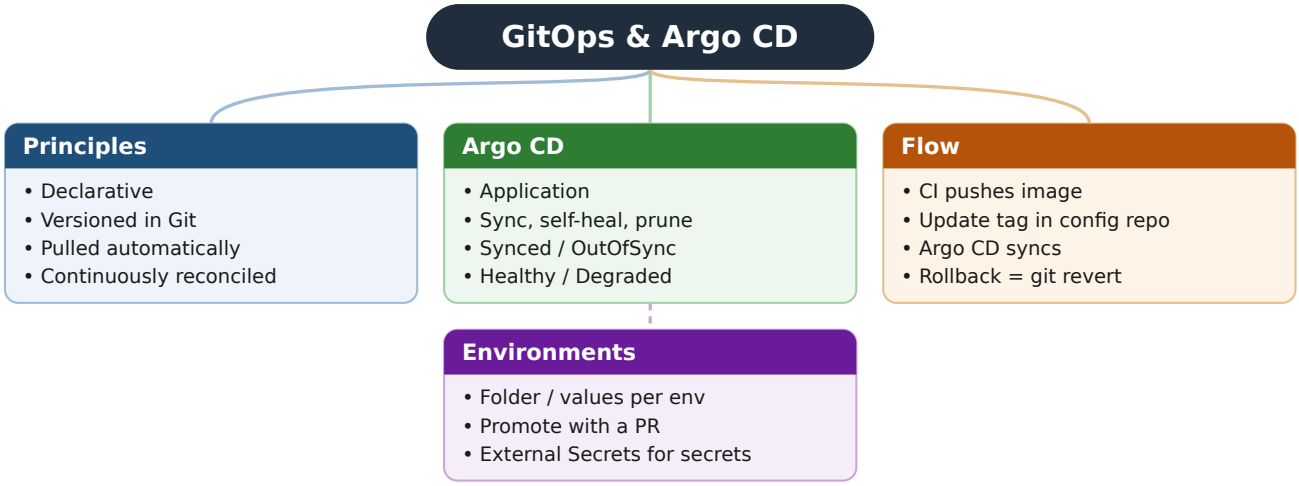
15. GitOps & Argo CD

In this section: 15.1 GitOps principles · 15.2 Argo CD



15.1 GitOps principles

1. **Declarative:** the whole desired state is described in files (YAML/Helm/Kustomize).
2. **Versioned in Git:** Git is the single source of truth, with history and review.
3. **Pulled automatically:** an agent in the cluster pulls changes from Git (no `kubectl apply` from laptops or CI).
4. **Continuously reconciled:** the agent fixes any drift so the cluster always matches Git.

```
Developer → PR to app repo → CI builds & pushes image:1.5.0
          → CI (or a bot) updates image tag in the config repo (PR)
          → Argo CD sees the change in Git → syncs the cluster
Rollback  = git revert in the config repo
```

15.2 Argo CD

Term	Meaning
Application	Links a Git path (repo, branch, folder/chart) to a cluster + namespace
Sync	Apply Git's state to the cluster
Synced / OutOfSync	Cluster matches Git / differs
Healthy / Degraded	App's resources are working / failing
Auto-sync, self-heal, prune	Sync automatically, undo manual changes, delete removed resources
App of Apps / ApplicationSet	Manage many Applications from one place / generate them per environment or cluster

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: web-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<user>/platform-config
    targetRevision: main
    path: apps/web/overlays/dev          # or a Helm chart path with values
  destination:
    server: https://kubernetes.default.svc
    namespace: dev
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

Command	What it does
`kubectl create ns argocd` + install manifest / Helm chart	Install Argo CD
`kubectl port-forward svc/argocd-server -n argocd 8080:443`	Open the UI
`argocd login localhost:8080`	CLI login
`argocd app list`, `argocd app sync web-dev`, `argocd app history web-dev`	Manage apps

Environments: one folder (or values file) per environment; promote from dev → prod with a pull request. **Secrets in GitOps:** never plain Secrets in Git. Use External Secrets Operator (reads from AWS Secrets Manager) or Sealed Secrets.

REMEMBER

- **Git is the source of truth.** Nobody runs `kubectl apply` on production by hand.